Electronic Companion to "Simultaneous Group-Envelope Bounds for Γ-Robust Multiple-Choice Knapsack Problems"

Eric Shao

Department of Mathematics, ETH Zürich, ershao@student.ethz.ch

August 2026

Comparator formulation, exact-integration audit, statistical protocol, generators, and reproducibility record.

A Comparator and exact-integration implementation

The main manuscript states the bounded-threshold group-clique comparator and proves its formal relationship to the minimax bound. Both of its constraint matrices are passed to HiGHS in sparse CSR form. An ambiguous generic HiGHS status is never accepted as an upper bound: the code retries dual simplex and interior point and accepts only optimizer-certified optimality or infeasibility.

The exact integration uses the same best-first interval queue for the envelope and clique variants. A globally robust HullRound solution initializes the incumbent. The root endpoints and midpoint are solved by the fixed-threshold branch-and-bound; afterward the algorithm bisects the active interval with largest retained bound. A singleton is solved exactly only when its interval remains capable of improving the incumbent. The complete-enumeration comparator uses the same fixed-threshold branch-and-bound ordered by fixed-threshold LP value, while compact SCIP uses the standard budgeted-uncertainty reformulation. The audit contains four families, 30 groups, three seeds, two balanced repetitions, and a five-second limit. Raw results, solver statuses, node counts, interval counts, objective agreement, and environment metadata are included in the evidence directory.

B Detailed protocol and statistical estimands

The statistical unit is an instance. Timing repetitions estimate the runtime of that instance and are not treated as independent observations. For primary instance $r \in \{1, \dots, R\}$, let t_r^C and t_r^Q be the medians of five wall-clock repetitions for the compressed and clique methods. The paired speedup is $s_r = t_r^Q / t_r^C$, and the reported aggregate is

$$\exp\left(\frac{1}{R}\sum_{r=1}^{R}\log s_r\right).$$

The percentile interval resamples seeds with replacement separately inside each family–size cell for 10,000 draws, preserving the factorial design. It is a design-stratified descriptive measure of variation across generated seeds, not a population-sampling interval for all robust MCKPs. No timing repetition enters this calculation.

The primary design has 60 instances: three sizes, four families, and five seeds. Robustness has 36 new instances: three configurations, four families, and three seeds at $n = 720$. Stress has eight new instances: two sizes, four families, and one seed. The kernel has 48 instances: four sizes, four families, and three seeds. Validation has 40 independently generated irregular cases. Two separately scoped nine-instance panels use UCI-calibrated semi-synthetic portfolios and published robust-knapsack coefficients, respectively. Thread environment variables were fixed to one for every phase.

C Exact instance generators

For auditability, we state the structured generator. Each group has a base option and $m - 1$ upgrades. Let q_k, $k = 1, \dots, m-1$, be equally spaced from 0.8 to 14, let $w_i \sim U[8, 8.4]$, and let G be the 25-point grid from 0.2 to 3.2. The base option has $(v_{i0}, a_{i0}, d_{i0}) = (w_i, b_f, 0)$, where b_f is 4.6, 4.1, 3.9, and 4.4 for dense frontier, correlated risk, near tie, and many breakpoints, respectively. Upgrade margins are

$$a_{ik} = b_f - q_k + \eta_{ik}, \qquad \eta_{ik} \sim N(0, 0.08^2).$$

Independent value noises give

$$v_{ik} = \max\{0, w_i + h_f(q_k) + \epsilon_{ik}\},$$

with $(h_f(q), \operatorname{sd}\epsilon)$ equal to $(8\sqrt{q}, 0.35)$, $(3.15q, 0.75)$, $(2.25q, 1.65)$, and $(7.5\sqrt{q}, 0.45)$ in the same family order. With zero-based group index i, deviations are

$$\begin{aligned}
d_{ik}^{\text{dense}} &= G[(7i + 3k + s) \bmod 25], \\
d_{ik}^{\text{corr}} &= \operatorname*{nearest}_{g \in G}\{0.30 + 0.19q_k + \zeta_{ik}\}, \quad \zeta_{ik} \sim N(0, 0.08^2), \\
d_{ik}^{\text{tie}} &= G[(5i + k + s) \bmod 25], \\
d_{ik}^{\text{break}} &= 0.15 + 0.003[i(m-1) + k] + 10^{-6}s,
\end{aligned}$$

where s is the reported seed. To reproduce the release exactly, the NumPy seed is the first 16 hexadecimal digits of `SHA256(`v4|family|n|m|Gamma|seed')` reduced modulo 2^{32}. This seed derivation is platform-independent; the released environment record and result files remain authoritative because numerical distribution routines can vary across library versions.

The irregular validation generator independently draws 8–20 groups and 2–10 options per group, signed Gaussian values, Gaussian margins, repeated deviations from $\{0, 0.25, 0.5, 1, 2, 4, 8\}$ with occasional perturbations below 0.05, and Γ uniformly from $\{0, \dots, n\}$. These cases target algebraic edge conditions rather than timing difficulty.

For the application-derived panel, the calibration reads the first 200,000 UCI records and aggregates

positive-quantity, positive-price observations by stock code. Products with fewer than eight retained observations are removed, leaving 192,451 contributing observations and 2,549 SKU aggregates. Price–volume quantiles determine five segment shares and reference medians. Within each generated portfolio, segment-calibrated lognormal prices and volumes combine with documented elasticity ranges; twelve psychologically rounded price choices induce objective, nominal-resource, and uncertainty coefficients. The released segment aggregate has SHA-256 digest 1ba04343eb853439f184a55dbd6ac3dbeac6f6438ef1b9dcc8b0173e3a40b7f6; the raw data are retrieved from the cited UCI record.

The published-coefficient panel reads the CC-BY robust-knapsack archive of Gersing et al. (2022), verified by SHA-256 digest 8571b3e545607415a38a39dc506b21bd891b6a22ce252e42a1622a5a5f451818. For each binary item, the parser creates an unselected and selected option using the exact two-option reduction stated in the main paper. The source nominal profit, weight, Γ, and deviation coefficient are otherwise unchanged. Because the source deviations perturb objective coefficients whereas the model studied here perturbs a resource row, this panel is labeled coefficient transfer throughout; its purpose is generator independence, not source-model replication.

Table C.1. Published robust-knapsack coefficient-transfer panel. Times are instance medians over two balanced repetitions. Source coefficients and budgets are retained, while deviations are transferred from the source uncertain objective to the resource constraint.

Binary items	Cases	Envelope (s)	Clique (s)	Geometric speedup	Wins
1,000	3	0.987	1.565	1.65	3/3
5,000	3	4.137	10.752	2.45	3/3
10,000	3	8.375	27.172	3.07	3/3

D Reproducibility checklist

The supplement contains the protocol and digest, phase-specific environments, instance-level and raw timing CSVs, summaries with gates, exact-epigraph certificate checks, unit tests for every algebraic and certificate invariant, and a generator for all numerical manuscript artifacts and their hash manifest. Kernel storage means persistent numerical-array bytes rather than process RSS; comparator matrix storage is measured from sparse CSR arrays; the two-repetition stress and published-coefficient panels are descriptive.

References

Álvarez-Miranda, E., Ljubić, I., and Toth, P. (2013). A note on the Bertsimas & Sim algorithm for robust combinatorial optimization problems. *4OR*, 11, 349–360. doi:10.1007/s10288-013-0231-6.

Atamtürk, A. (2006). Strong formulations of robust mixed 0–1 programming. *Mathematical Programming*, 108, 235–250. doi:10.1007/s10107-006-0709-5.

Bertsimas, D. and Sim, M. (2003). Robust discrete optimization and network flows. *Mathematical Programming*, 98, 49–71. doi:10.1007/s10107-003-0396-4.

Bertsimas, D. and Sim, M. (2004). The price of robustness. *Operations Research*, 52, 35–53. doi:10.1287/opre.1030.0065.

Büsing, C., Gersing, T., and Koster, A. M. C. A. (2023). A branch and bound algorithm for robust binary optimization with budget uncertainty. *Mathematical Programming Computation*, 15, 269–326. doi:10.1007/s12532-022-00232-2.

Caserta, M. and Voß, S. (2019). The robust multiple-choice multidimensional knapsack problem. *Omega*, 86, 16–27. doi:10.1016/j.omega.2018.06.014.

Chen, D. (2015). Online Retail [Dataset]. *UCI Machine Learning Repository*. doi:10.24432/C5BW33.

Dyer, M. E. (1984). An $O(n)$ algorithm for the multiple-choice knapsack linear program. *Mathematical Programming*, 29, 57–63. doi:10.1007/BF02591729.

Fischetti, M. and Monaci, M. (2012). Cutting plane versus compact formulations for uncertain (integer) linear programs. *Mathematical Programming Computation*, 4, 239–273. doi:10.1007/s12532-012-0039-y.

Gersing, T., Büsing, C., and Koster, A. M. C. A. (2022). Benchmark instances for robust combinatorial optimization with budgeted uncertainty. *Zenodo*. doi:10.5281/zenodo.7419028.

Hansknecht, C., Richter, A., and Stiller, S. (2018). Fast robust shortest path computations. *OASIcs ATMOS*, 65, 5:1–5:21. doi:10.4230/OASIcs.ATMOS.2018.5.

Joung, S. and Park, K. (2021). Robust mixed 0–1 programming and submodularity. *INFORMS Journal on Optimization*, 3, 183–199. doi:10.1287/ijoo.2019.0042.

Joung, S., Oh, S., and Lee, K. (2023). Comparative analysis of linear programming relaxations for the robust knapsack problem. *Annals of Operations Research*, 323, 65–78. doi:10.1007/s10479-022-05161-w.

Lee, T. and Kwon, C. (2014). A short note on robust combinatorial optimization problems with cardinality constrained uncertainty. *4OR*, 12, 373–378. doi:10.1007/s10288-014-0270-7.

Monaci, M., Pferschy, U., and Serafini, P. (2013). Exact solution of the robust knapsack problem. *Computers & Operations Research*, 40, 2625–2631. doi:10.1016/j.cor.2013.05.005.

Sinha, P. and Zoltners, A. A. (1979). The multiple-choice knapsack problem. *Operations Research*, 27, 503–515. doi:10.1287/opre.27.3.503.

Szkaliczki, T. (2025). Solution methods for the multiple-choice knapsack problem and their applications. *Mathematics*, 13, 1097. doi:10.3390/math13071097.

Zemel, E. (1980). The linear multiple choice knapsack problem. *Operations Research*, 28, 1412–1423. doi:10.1287/opre.28.6.1412.